

# Beyond the Binary Barrier

Extending RetroWrite with memcpy-safe sanitisation,  
standalone coverage, and native function tracing.

---

## GROUP

Chirag Suthar · 2025MCS2105

Haleel Sada · 2025MCS2741

## REFERENCE

Dinesh et al., RetroWrite,  
IEEE S&P 2020.

Instructor · Prof. Huzur Saran

April 2026

# Most bugs are memory bugs.

---

# 70%

of CVEs are memory-safety issues.

1

## Closed-Source Targets

Browsers, firmware, games ship as stripped ELF / PE binaries — no source to instrument.

2

## Dynamic Tools are Too Slow

QEMU 10–100× overhead, Valgrind up to 300× — unusable for real fuzzing.

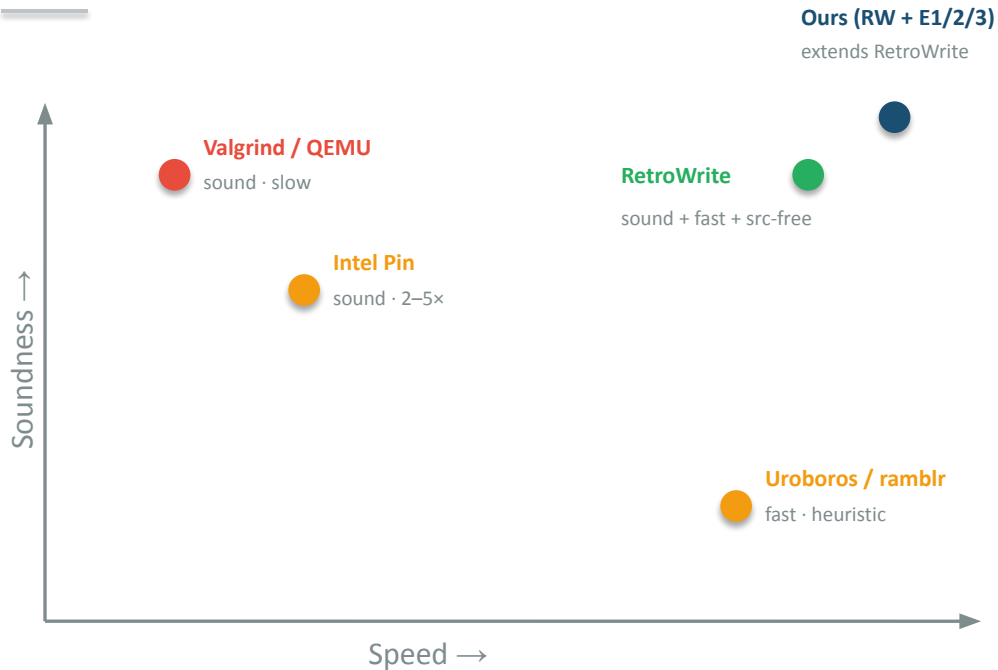
3

## Static Rewriters Guess

Uroboros, ramblr rely on heuristics that break on real binaries — reassembled code crashes.

We close three gaps in RetroWrite — memcpy sanitisation, x64 coverage, and a native tracer.

# The classical trilemma.



## THE INSIGHT

**Relocations in PIE ELF's  
label which bytes are  
addresses — exactly.**

No guessing. Fast and sound at once. Our passes inherit this guarantee.

# How binary rewriting works.

---



## PRIOR STATIC REWRITERS

**Guess:** is 0x401050 an address or a constant?

Heuristics break on real ELF's — reassembled binaries crash.

## RETROWRITE

**.rela.dyn** is a ground-truth oracle.

Every address-valued immediate is labelled — symbolisation is exact.

# ASan and AFL, in a sentence.

---

## ADDRESS SANITIZER

**Shadow memory catches out-of-bounds and use-after-free.**

- One shadow byte per 8 application bytes
- Every load/store calls `asan_load1/4/8`
- Red-zones around heap and stack allocations

## AMERICAN FUZZY LOP

**Edge-hash coverage guides input mutation.**

- 64 KiB bitmap indexed by  $\text{cur} \oplus \text{prev}$
- Small stub at every basic-block entry
- New bitmap bit = new test case saved

Both normally need source. RetroWrite brings them to stripped binaries.

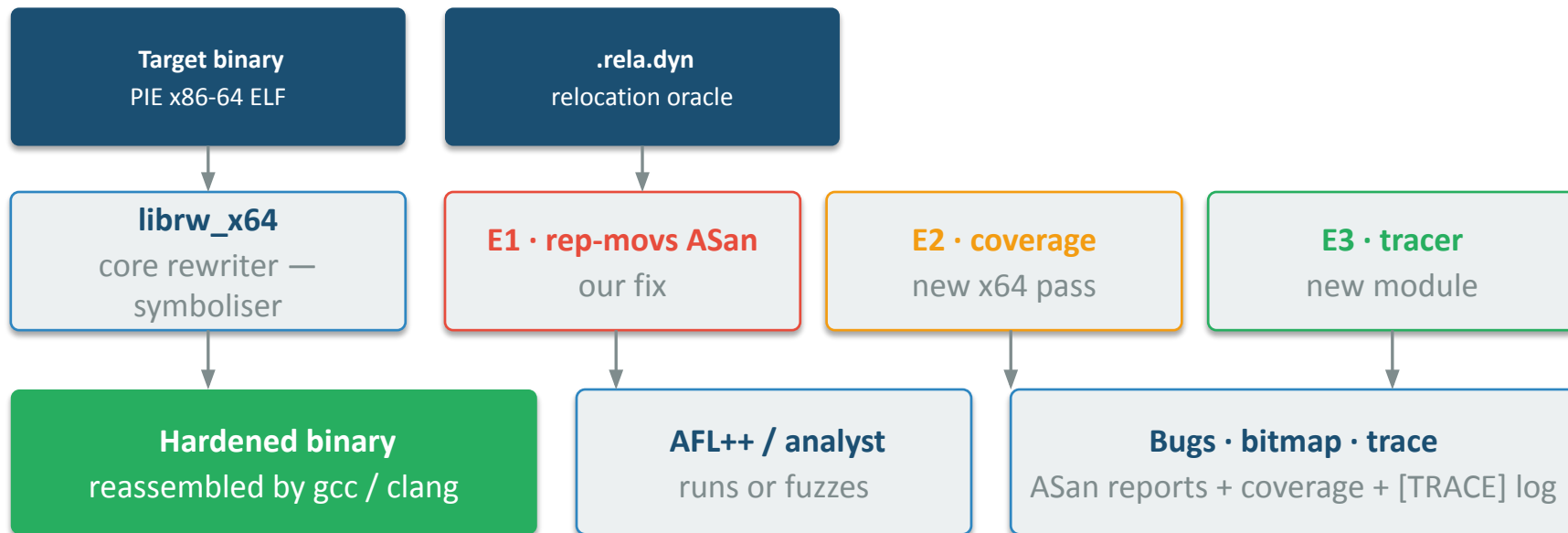
## Related work.

---

| Tool               | Sound | Fast    | Src-free | Notes                                |
|--------------------|-------|---------|----------|--------------------------------------|
| QEMU               | yes   | 10–100× | yes      | too slow for fuzzing                 |
| Valgrind           | yes   | 20–300× | yes      | heavyweight                          |
| Intel Pin          | yes   | 2–5×    | yes      | proprietary                          |
| Uroboros / ramblr  | part. | fast    | yes      | heuristic symbolisation              |
| DynInst            | part. | fast    | yes      | unsafe on stripped ELF               |
| E9Patch            | yes   | fast    | yes      | patch-style, limited semantics       |
| RetroWrite         | yes   | fast    | yes      | no rep-movs · no x64 cov · no tracer |
| Ours (RW + E1/2/3) | yes   | fast    | yes      | rep sanitised + coverage + tracer    |

Our row is last — no prior tool offers sound + fast + source-free + rep-movs sanitisation + coverage + tracing.

# System model.



All three passes live in `rwtools_x64/` and obey RetroWrite's `-m <module>` CLI contract.

# Threat model

---

## Attacker

- Arbitrary input (stdin/network)
- Cannot patch binary or kernel
- Assumes ASLR, NX are ON
- Goal: Control hijack or DoS

## Defender

- Only has stripped PIE ELF
- Adds RetroWrite instrumentation
- Constraint: Near-native speed
- Wants: Find bugs pre-deploy

## Out of Scope

- Pure logic/auth bugs
- Side-channels / timing
- Non-PIE binaries
- Kernel-level compromise

Soundness holds only for PIE binaries with intact relocations.



## Three extensions.

---

# E1

### rep-movs / rep-stos ASan

---

Four shadow probes catch memcpy overflows that RetroWrite silently missed.

`DISABLE_REP_FIX=1`

# E2

### Basic-block coverage

---

AFL-compatible bitmap via raw mmap no AFL runtime required.

`DISABLE_COVERAGE=1`

# E3

### Function-call tracer

---

Call-site trampolines log every entry to a circular buffer.

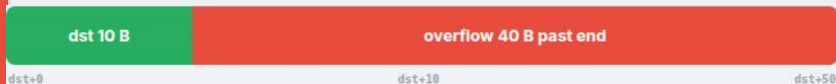
`DISABLE_TRACE=1`

Each pass drops in via `python3 retrowrite -m <mod>.`

# E1 · Malloc safety, finally.

## BEFORE

### Silent memcpy overflow



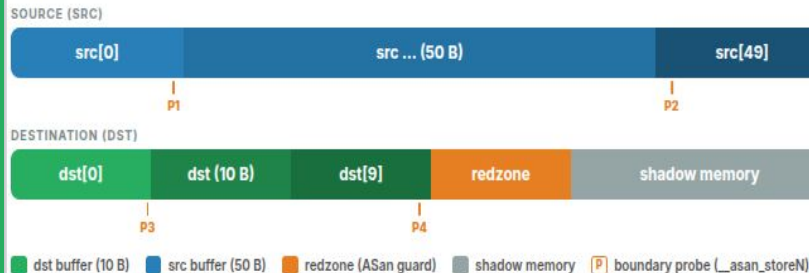
rep movsb copies 50 B into a 10 B buffer.

```
if mnemonic.startsWith("rep stos"):
    pass # XXX: TODO for more accurate check
# rep movs — not handled at all
```

Runtime: SEGV — silent corruption.

## AFTER

### Four boundary probes



Runtime result: ASan READ of size 1 — precise, actionable report

Each probe hits shadow memory and can fault safely.

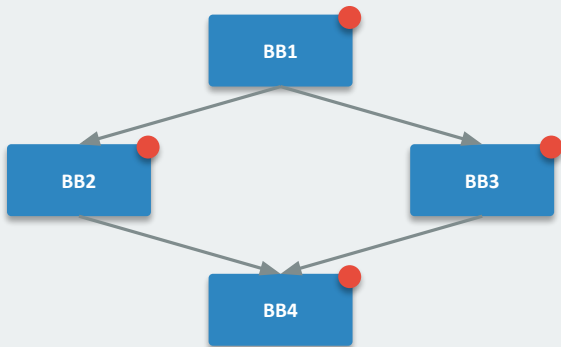
Runtime: AddressSanitizer READ of size 1.

## E2 and E3 · Coverage + tracing.

### E2

#### Basic-block coverage

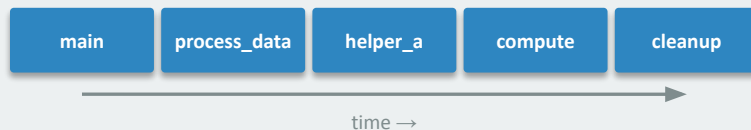
Red mark = edge-hash trampoline at BB entry.



bitmap[cur  $\oplus$  prev]++ · 64 KiB via raw mmap.

### E3

#### Function-call tracer



```
subq $8, %rsp          ; 16-B re-align
pushq %rdi ... %r11     ; save 9 regs
leaq .LTRACE_NAME(%rip),%rdi
callq __trace_log_entry
```

Both passes use the same trampoline pattern — spill, call, restore.

## Midterm results.

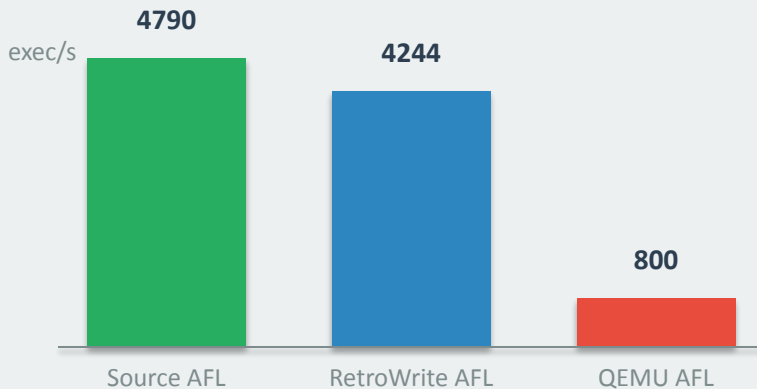
### BUG DETECTION

#### All four classes caught

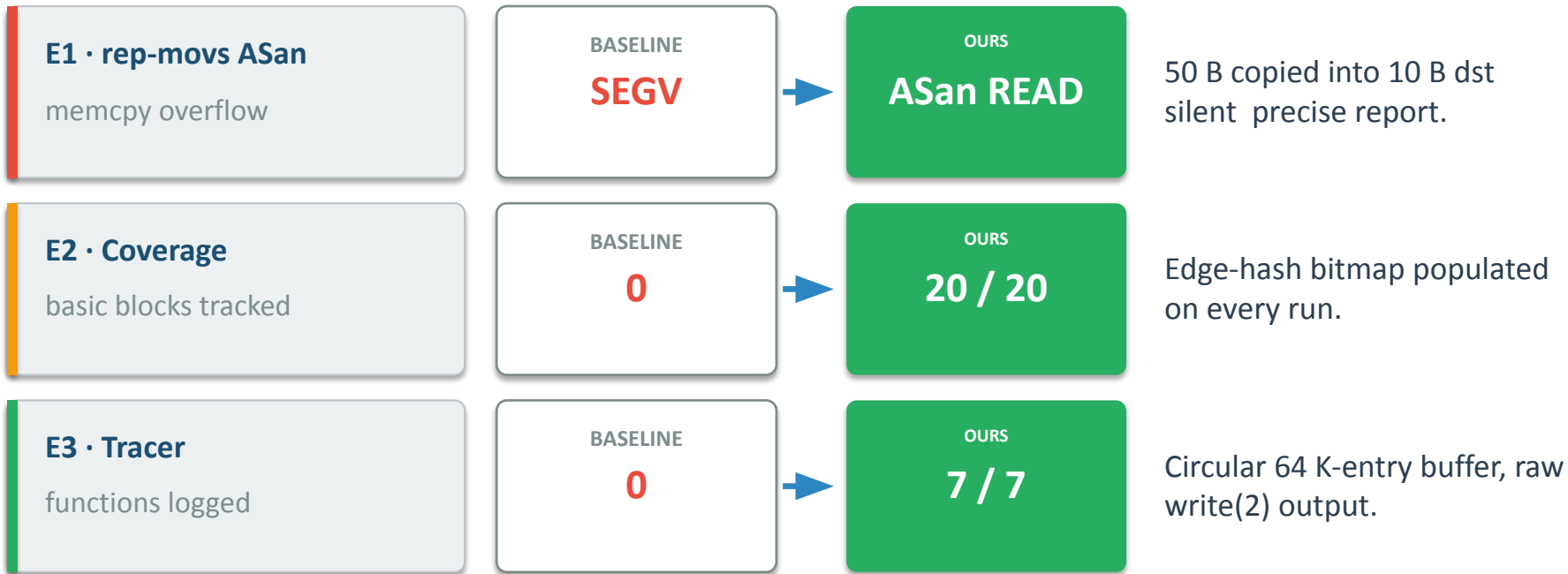
| Test case          | State       |
|--------------------|-------------|
| Heap out-of-bounds | silent      |
| Use-after-free     | silent      |
| Stack overflow     | SEGV        |
| Double free        | glibc abort |

### AFL THROUGHPUT

#### 88.6% of source speed



## Endterm results.



All three passes are toggleable via `DISABLE_REP_FIX` / `DISABLE_COVERAGE` / `DISABLE_TRACE`.

## Current limitations & future work.

---

### CURRENT LIMITATIONS

- PIE-only — non-PIE has no relocations
- No multi-threaded target support
- Tracer buffer overflows on very long runs
- No per-object redzones (no DWARF)

### Probabilistic Disassembly

Non-PIE binaries.

### Prologue / Epilogue Pass

Stack canaries.

### DWARF-aware Boundaries

Per-object redzones.

### Indirect-call Matching

Forward-edge CFI.

## Summary.

---

### Reproduced the paper

bzip2 identical — 88.6% of source-AFL speed — 5.3× faster than QEMU.

### Closed TODOs in upstream

x64 coverage and native tracer — all fully toggleable via env vars.

### Added coverage + tracer

rep movs / stos now instrumented — silent SEGV becomes a precise report.

Relocation-guided rewriting is an extensible platform.

# References ·

---

- [1] S. Dinesh, N. Burow, D. Xu, and M. Payer, "RetroWrite: Statically Instrumenting COTS Binaries for Fuzzing and Sanitization," OPAL (Open@LaTrobe), La Trobe University, May 2020. <https://doi.org/10.1109/sp40000.2020.00009>
- [2] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "AddressSanitizer: A Fast Address Sanity Checker," Proc. USENIX Annual Technical Conference (ATC), 2012, pp. 309–318. <https://www.usenix.org/conference/atc12/technical-sessions/presentation/serebryany>
- [3] M. Zalewski, "American Fuzzy Lop (AFL)," 2017. <https://lcamtuf.coredump.cx/afl/>
- [4] F. Bellard, "QEMU, a Fast and Portable Dynamic Translator," Proc. USENIX Annual Technical Conference (ATC), 2005, pp. 41–46. <https://www.usenix.org/conference/2005-usenix-annual-technical-conference/qemu-fast-and-portable-dynamic-translator>
- [5] N. Nethercote and J. Seward, "Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation," Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI), 2007, pp. 89–100. <https://valgrind.org/docs/valgrind2007.pdf>
- [6] C.-K. Luk et al., "Pin: Building Customized Program Analysis Tools with Dynamic Instrumentation," ACM SIGPLAN Notices, vol. 40, no. 6, pp. 190–200, Jun. 2005. <https://doi.org/10.1145/1064978.1065034>
- [7] D. Bruening, T. Garnett, and S. Amarasinghe, "An Infrastructure for Adaptive Dynamic Optimization," Proc. Int. Symp. Code Generation and Optimization (CGO), 2003, pp. 265–275. <https://commit.csail.mit.edu/papers/2003/RIO-adaptive-CGO03.pdf>
- [8] S. Wang, P. Wang, and D. Wu, "Reassembleable Disassembling," Proc. USENIX Security Symposium, 2015, pp. 627–642. <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/wang-shuai>
- [9] R. Wang et al., "Ramblr: Making Reassembly Great Again," Proc. Network and Distributed System Security Symposium (NDSS), 2017. <https://www.ndss-symposium.org/ndss2017/ndss-2017-programme/ramblr-making-reassembly-great-again/>
- [10] A. R. Bernat and B. P. Miller, "Anywhere, Any-Time Binary Instrumentation," Proc. ACM SIGPLAN Workshop on Program Analysis for Software Tools and Engineering (PASTE), Sep. 2011, pp. 9–16. <https://doi.org/10.1145/2024569.2024572>
- [11] G. J. Duck, X. Gao, and A. Roychoudhury, "Binary Rewriting without Control Flow Recovery," Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI), Jun. 2020, pp. 151–163. <https://doi.org/10.1145/3385412.3385972>
- [12] "What is Address Space Layout Randomization (ASLR)?" ReasonLabs Cyberpedia, 2023. [https://cyberpedia.reasonlabs.com/EN/address%20space%20layout%20randomization%20\(aslr\).html](https://cyberpedia.reasonlabs.com/EN/address%20space%20layout%20randomization%20(aslr).html)



# References ·

---

- [13] N. A. Quynh, "Capstone: Next-Generation Disassembly Framework," Proc. Black Hat USA, 2014. <https://www.capstone-engine.org/>
- [14] MITRE Corp., "CWE-120: Buffer Copy without Checking Size of Input." <https://cwe.mitre.org/data/definitions/120.html>
- [15] MITRE Corp., "CWE-121: Stack-Based Buffer Overflow." <https://cwe.mitre.org/data/definitions/121.html>
- [16] S. Rostedt, "ftrace: Function Tracer for Linux." <https://www.kernel.org/doc/Documentation/trace/ftrace.txt>
- [17] J. Weidendorfer, "Callgrind: A Call-Graph Generating Cache Profiler," Valgrind Project, 2004.
- [18] J. Seward, "bzip2 and libbzip2, version 1.0.8," 2019. <https://sourceware.org/bzip2/>
- [19] K. Miller et al., "Probabilistic Disassembly," Proc. Int. Conf. Software Engineering (ICSE), 2019, pp. 1187–1198. <https://doi.org/10.1109/ICSE.2019.00123>
- [20] P. Chen and H. Chen, "Angora: Efficient Fuzzing by Principled Search," Proc. IEEE Symp. Security & Privacy (S&P), 2018, pp. 711–725. <https://doi.org/10.1109/SP.2018.00046>
- [21] Intel Corp., "Control-flow Enforcement Technology Specification," 2019.  
<https://www.intel.com/content/www/us/en/developer/articles/technical/control-flow-enforcement-technology.html>
- [22] L. Szekeres et al., "SoK: Eternal War in Memory," Proc. IEEE Symp. Security & Privacy (S&P), 2013, pp. 48–62. <https://doi.org/10.1109/SP.2013.13>

**Thank You**